Hoja rápida · Bases de datos (JDBC + SQLite) y ficheros en Java

Acceso a Datos · 2º DAM · Repaso para el examen del 25 nov 2026.

1. Estructura por capas

Capa	Clase	Responsabilidad
Modelo	`Tarea`	Atributos `private`, constructores, getters/setters, `toString`. No sabe nada de la base de datos.
Datos (DAO)	`TareaDAO`	Todo el SQL. Métodos `static` con `throws SQLException`. No imprime ni captura.
Interfaz	`Main`	Menú, lectura de teclado, `try/catch` y mensajes al usuario.

2. Molde de un método DAO (JDBC)

1. Escribir el SQL con `?` donde van los datos (constante `private static final`).
2. Abrir `Connection` y `PreparedStatement` en un `try-with-resources`.
3. Rellenar los `?`: `setString(1, …)`, `setInt(2, …)`. **Se numeran desde 1.**
4. Ejecutar y, si es un `SELECT`, recorrer el `ResultSet`.

Escribir: INSERT / UPDATE / DELETE

```
public static boolean eliminar(int id)
        throws SQLException {
    boolean ok = false;
    try (Connection c =
             DriverManager.getConnection(URL);
         PreparedStatement ps =
             c.prepareStatement(SQL_ELIMINAR))
{
        ps.setInt(1, id);
        ok = ps.executeUpdate() > 0;
    }
    return ok;
}
```

Leer: SELECT

```
public static ArrayList<Tarea> listarTodos()
        throws SQLException {
    ArrayList<Tarea> lista = new ArrayList<>();
    try (Connection c =
             DriverManager.getConnection(URL);
         PreparedStatement ps =
             c.prepareStatement(SQL_LISTAR)) {
        try (ResultSet rs = ps.executeQuery())
{
            while (rs.next()) {
                lista.add(mapear(rs));
            }
        }
    }
    return lista;
}
```

```
private static Tarea mapear(ResultSet rs) throws SQLException {
    return new Tarea(rs.getInt("id"), rs.getString("titulo"), rs.getBoolean("hecha"));
}
```

Método	Se usa con	Devuelve
`executeQuery()`	`SELECT`	`ResultSet`: se recorre con `rs.next()`
`executeUpdate()`	`CREATE`, `INSERT`, `UPDATE`, `DELETE`	`int`: filas afectadas (`> 0` = éxito)

Orden de un acceso JDBC: abrir conexión → preparar el `PreparedStatement` → rellenar `?` → ejecutar → leer el `ResultSet`. Se cierra en orden inverso, y lo hace el `try-with-resources`.

3. SQL que se usa en los ejercicios

```
CREATE TABLE IF NOT EXISTS tarea(id INTEGER PRIMARY KEY AUTOINCREMENT,
                                 titulo TEXT NOT NULL, hecha INTEGER NOT NULL)
INSERT INTO tarea(titulo, hecha) VALUES(?, ?)            -- sin id: lo genera SQLite
SELECT * FROM tarea WHERE codigo = ?
SELECT * FROM producto WHERE LOWER(nombre) LIKE LOWER(?) -- setString(1, "%" + texto + "%")
UPDATE producto SET precio = ? WHERE codigo = ?
DELETE FROM tarea WHERE id = ?
SELECT COUNT(codigo), AVG(precio), MAX(precio), MIN(precio) FROM producto
SELECT categoria, COUNT(codigo) FROM producto GROUP BY categoria
```

- URL de SQLite: `jdbc:sqlite:archivo.db` (se crea solo al conectar).
- SQLite no tiene booleano: `hecha` es `INTEGER` (0/1). `setBoolean` y `getBoolean` convierten.
- Una clave primaria repetida lanza `SQLException`: se captura en quien llama al DAO.
- Los DAOs **propagan** la excepción (`throws SQLException`); `Main` y `GestorFicheros` deciden qué mostrar.

Errores que ya salieron (revisar antes de entregar)

Error	Qué provoca
`CREATE TABLE` sin nombre de tabla, sin paréntesis o con `WHERE`	`SQLException` al arrancar
`mapear` con el constructor sin id, o constructor que no asigna `this.id`	Todos los ids salen como 0 y `eliminar` no sirve
`listarTodos` sin `return lista`	El resultado se pierde
`ORDER BY` por una columna que no existe	`SQLException` al listar
Número de `?` distinto del de `setX`	`SQLException` o columna sin rellenar

4. Ficheros de texto: lectura y escritura

Escribir con `BufferedWriter`

```
try (BufferedWriter bw = new BufferedWriter(
        new FileWriter(archivo))) {
    for (Producto p : lista) {
        bw.write(String.format(Locale.US,
            "%s;%s;%s;%.2f;%d", p.getCodigo(),
            p.getNombre(), p.getCategoria(),
            p.getPrecio(), p.getStock()));
        bw.newLine();
    }
} catch (IOException e) { ... }
```

- `new FileWriter(f)` **sobrescribe**.
- `new FileWriter(f, true)` **añade** al final.
- `Locale.US` obliga a punto decimal. Sin él, en un sistema en español el decimal sale con coma y corrompe el CSV.

Leer con `BufferedReader`

```
try (BufferedReader br = new BufferedReader(
        new FileReader(archivo))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        String[] campos = linea.split(";");
        if (campos.length != 5) {
            // línea mal formada: saltarla
            continue;
        }
        ...
    }
} catch (IOException e) { ... }
```

- `readLine()` devuelve `null` al final.
- `read()` devuelve `-1` al final.
- `File.exists()` comprueba si el fichero existe.

Importar una línea sin que un fallo aborte todo el bucle

```
try {
    Producto p = new Producto(campos[0], campos[1], campos[2],
                              Double.parseDouble(campos[3]), Integer.parseInt(campos[4]));
    ProductoDAO.insertar(p);
    insertados++;
} catch (NumberFormatException e1) { saltadas++; }   // subclase PRIMERO
  catch (IllegalArgumentException e2) { saltadas++; } // la superclase después
  catch (SQLException e3) { saltadas++; }             // un fallo del DAO no aborta el resto
// al terminar el while: imprimir el resumen (insertadas, actualizadas, saltadas)
```

- `NumberFormatException` es subclase de `IllegalArgumentException`: el `catch` de la subclase va **antes**, o no compila.
- Si el `try` ya no lanza una excepción comprobada (`SQLException`), capturarla en un multi-catch es un error de compilación: *exception is never thrown in body of corresponding try statement*.

API moderna: `java.nio.file`

Quiero…	Código
Leer todo como texto	`Files.readString(ruta)`
Leer todas las líneas	`List<String> l = Files.readAllLines(ruta);`
Crear o **sobrescribir**	`Files.writeString(ruta, texto)`
Añadir (crea si no existe)	`Files.writeString(ruta, texto, StandardOpenOption.CREATE, StandardOpenOption.APPEND)`
Crear solo si **no existe**	`StandardOpenOption.CREATE_NEW` (falla si existe)

- `CREATE` **solo**, sobre un fichero existente, **no vacía**: escribe encima y deja restos.
- `APPEND` solo, sobre un fichero que no existe, lanza `NoSuchFileException`.
- Nunca mezclar `APPEND` con `TRUNCATE_EXISTING`.
- Ruta: `Path.of("notas.txt")`. Cuidado con comillas sueltas dentro de la cadena.

5. Teclado con `Scanner` y menús

- **Lee todo con `sc.nextLine()`** y convierte después: `Integer.parseInt`, `Double.parseDouble`, `Boolean.parseBoolean`.
- No mezcles `nextInt()` / `nextBoolean()` con `nextLine()`: dejan el salto de línea en el buffer y el siguiente `nextLine()` lee una cadena vacía.
- La variable del menú es **local** y se inicializa a `-1` (con 0, un primer valor inválido cierra el programa).
- Protege cada `parseInt` / `parseDouble` con `catch (NumberFormatException e)`: avisa y vuelve al menú.
- Cada llamada al DAO va en su propio `try/catch (SQLException e)`.

6. Lo que falta por practicar

- **Semana 26-30 oct:** serialización de objetos, JSON y XML (guardar y leer la lista de `Tarea`).
- **Transacciones:** `conexion.setAutoCommit(false)`, varias sentencias, `commit()` y, en el `catch`, `rollback()`.
- **JOIN:** una consulta con dos tablas relacionadas, mapeada a objetos.
- **Más:** JUnit para el DAO, y la UD2 (ORM con JPA/Hibernate y NoSQL).

Basada en los ejercicios `jdbc-sqlite` (`gestorProductos`, `gestorTareas`) y `AccesoADatos`. Enunciado completo del gestor de tareas: `ENUNCIADO_gestorTareas.md`.